

Consistency and Large Sample Behavior

Math 742

2026-02-18

Intro

Sometimes when we have only a small sample of data, results can be far from what is really going on. Thumbrules in textbooks and medical literature based on small samples can often be very wrong. Many of the traditional techniques you learned in your intro to statistics courses are often based on small samples. Often they are misleading or even sometimes flat out wrong.

Consider this example.

When we have an iid sample of size n from a continuous population distribution with median $\tilde{\mu}$, asymptotic theory tells us that, for large n , the sample median is approximately normally distributed:

Sample Median $= N(\tilde{\mu}, \pi^2/4n)$. That is, the asymptotic variance is $\pi^2/4n$.

This statement is true for every continuous distribution (even ones with no mean or variance, like the Cauchy). The catch is the phrase “for large n .” The example below shows just how large “large” sometimes has to be, and how badly the normal approximation can fail for sample sizes we routinely see in practice ($n = 25, 100$, or even 1000).

In short: the central limit theorem for the median is mathematically correct, but the speed of convergence can be excruciatingly slow for some distributions. That is the main reason we cannot blindly trust asymptotic formulas in finite samples and why understanding large-sample behavior is essential.

```
set.seed(123)
B <- 200000L

# Cauchy median - correct asymptotic variance is pi^2/(4n)
cat("Asymptotic variance of the sample median from Cauchy(0,1)\n\n")

## Asymptotic variance of the sample median from Cauchy(0,1)

for (n in c(5,15, 25,100,500)) {
  medians <- replicate(B, median(rcauchy(n)))
  mc_var <- var(medians)
  asymp_var <- pi^2 / (4 * n)                # exact asymptotic formula

  cat(sprintf("n = %5d  Monte Carlo variance = %.6f  Asymptotic = %.6f  Ratio MC/asymp = %.3f\n",
              n, mc_var, asymp_var, mc_var / asymp_var))
}
```

## n =	5	Monte Carlo variance = 1.205450	Asymptotic = 0.493480	Ratio MC/asymp = 2.443
## n =	15	Monte Carlo variance = 0.205356	Asymptotic = 0.164493	Ratio MC/asymp = 1.248
## n =	25	Monte Carlo variance = 0.112150	Asymptotic = 0.098696	Ratio MC/asymp = 1.136
## n =	100	Monte Carlo variance = 0.025195	Asymptotic = 0.024674	Ratio MC/asymp = 1.021
## n =	500	Monte Carlo variance = 0.004965	Asymptotic = 0.004935	Ratio MC/asymp = 1.006

- At $n = 25$ (the size of many real studies) the asymptotic formula is over by $\sim 13.6\%$.

- At $n = 100$ the asymptotic variance is over by 2.1% of the true variance.
- At $n = 500$ the normal approximation is almost perfect.

Consistency

The basic idea with consistency is that as the sample size grows, your estimator gets closer to the true parameter. The formal way to say this is that the estimator converges in probability to the true parameter.

The math that governs consistency:

- Law of Large Numbers: The sample averages converge to the expectations.
- Continuous Mapping: Convergence is preserved under continuous transformations.

If $\theta_n \rightarrow \theta_0$ and $g(\cdot)$ is continuous at θ_0 , then $g(\theta_n) \rightarrow g(\theta_0)$

- Identification (i.e. Identifiable $\implies$ 1-1): There is only one parameter value in the universe that is compatible with the distribution that actually generated the data.

$$\theta \neq \theta' \implies P_\theta \neq P_{\theta'}$$

Examples:

- Sample mean for Normal or any finite-mean distribution.
- Proportion $\hat{p}$ is consistent for p .
- MLE: generally consistent because the likelihood points toward the true parameter as sample grows (intuition, not regularity conditions).

Three crucial facts about consistency:

- Consistent estimators are eventually correct.
- Even biased estimators can be consistent (e.g., sample variance with $1/n$ factor).
- Consistency $\neq$ unbiasedness

Large Sample Intuition

Here is a demonstration of the Law of large numbers.

What we are seeing in the picture is that as n gets larger and larger, the sample mean converges in probability to the population mean.

$$\bar{X}_n \xrightarrow{p} \mu$$

or equivalently,

$$\lim_{n \rightarrow \infty} \mathbb{P}(|\bar{X}_n - \mu| > \varepsilon) = 0 \text{ for every } \varepsilon > 0$$

```
library(ggplot2)
library(patchwork)

set.seed(123)

# sample sizes
```

```

n_values <- c(5, 15, 30, 50, 100, 500)

# population parameters
mu <- 50
sigma <- 1

# number of simulations per sample size
B <- 5000

# --- Set a common x-axis range ---
# widest expected sampling distribution is for n = 5:
sd5 <- sigma / sqrt(5)
xmin <- mu - 4 * sd5
xmax <- mu + 4 * sd5

plots <- list()

for (i in seq_along(n_values)) {

  n <- n_values[i]

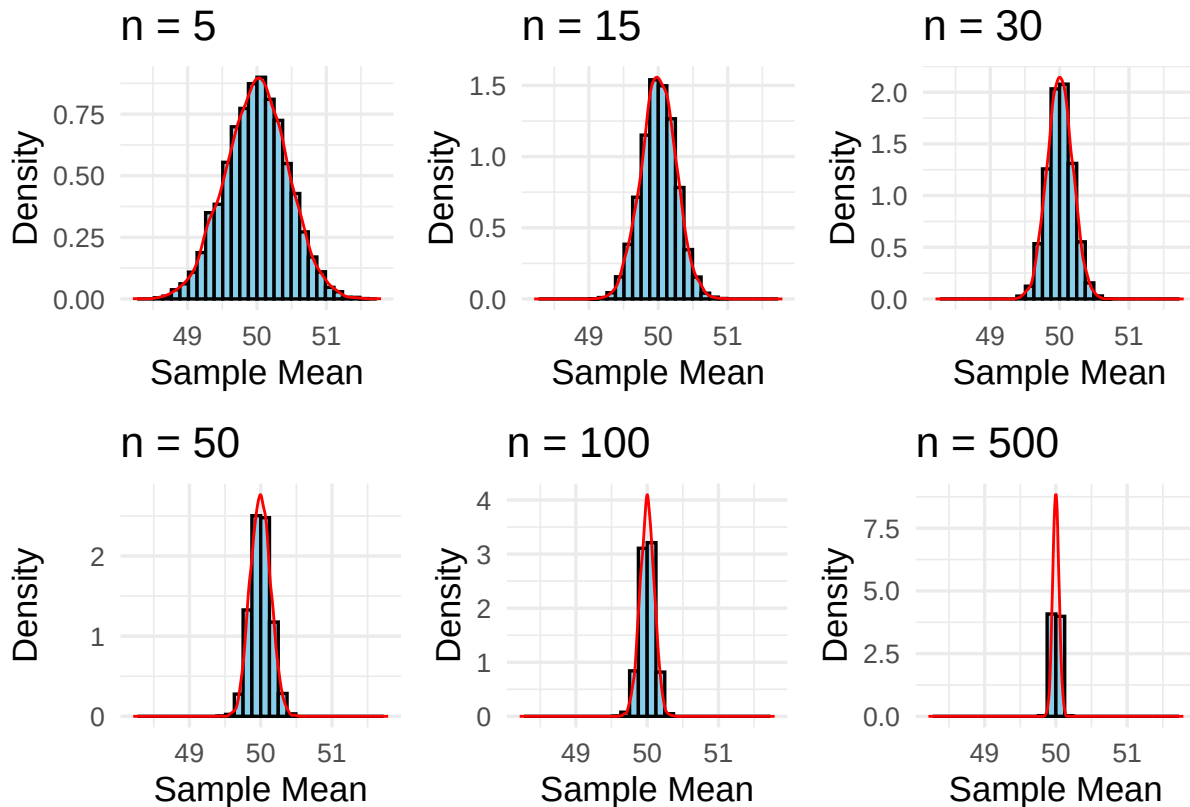
  # simulate B sample means
  sample_means <- replicate(B, mean(rnorm(n, mean = mu, sd = sigma)))
  df <- data.frame(mean = sample_means)

  # plot sampling distribution with fixed x-limits
  p <- ggplot(df, aes(x = mean)) +
    geom_histogram(aes(y = after_stat(density)),
                  bins = 30,
                  fill = "skyblue",
                  color = "black") +
    geom_density(color = "red", linewidth = 0.5) +
    xlim(xmin, xmax) +
    labs(title = paste("n =", n),
         x = "Sample Mean",
         y = "Density") +
    theme_minimal(base_size = 13)

  plots[[i]] <- p
}

# 2 x 3 grid using patchwork
(plots[[1]] | plots[[2]] | plots[[3]]) /
(plots[[4]] | plots[[5]] | plots[[6]])

```



So, what you are seeing is that as the sample size (n) increases, the probability that the sample mean deviates from the true mean by any positive amount ε goes to zero.

As far as the shape of the distribution goes, we are sampling from a normal distribution, so the distribution converges to a normal distribution. In fact though, it doesn't matter if we sample from a normal or some other distribution the distribution will still converge to a normal distribution.

Consider this demo. This time we sample from an exponential distribution with $\lambda = 2$.

```
set.seed(123)

# sample sizes
n_values <- c(5, 15, 30, 50, 100, 500)

# population distribution: Exponential with rate lambda=2
lambda <- 2
mu <- 1 / lambda           # mean = 0.5
sigma <- 1 / lambda        # sd = 0.5

# number of simulations
B <- 5000

# wide enough x-range based on smallest-n distribution
sd5 <- sigma / sqrt(5)
xmin <- mu - 4 * sd5
xmax <- mu + 4 * sd5

plots <- list()
```

```

for (i in seq_along(n_values)) {

  n <- n_values[i]

  # simulate B sample means from Exponential(lambda)
  sample_means <- replicate(B, mean(rexp(n, rate = lambda)))
  df <- data.frame(mean = sample_means)

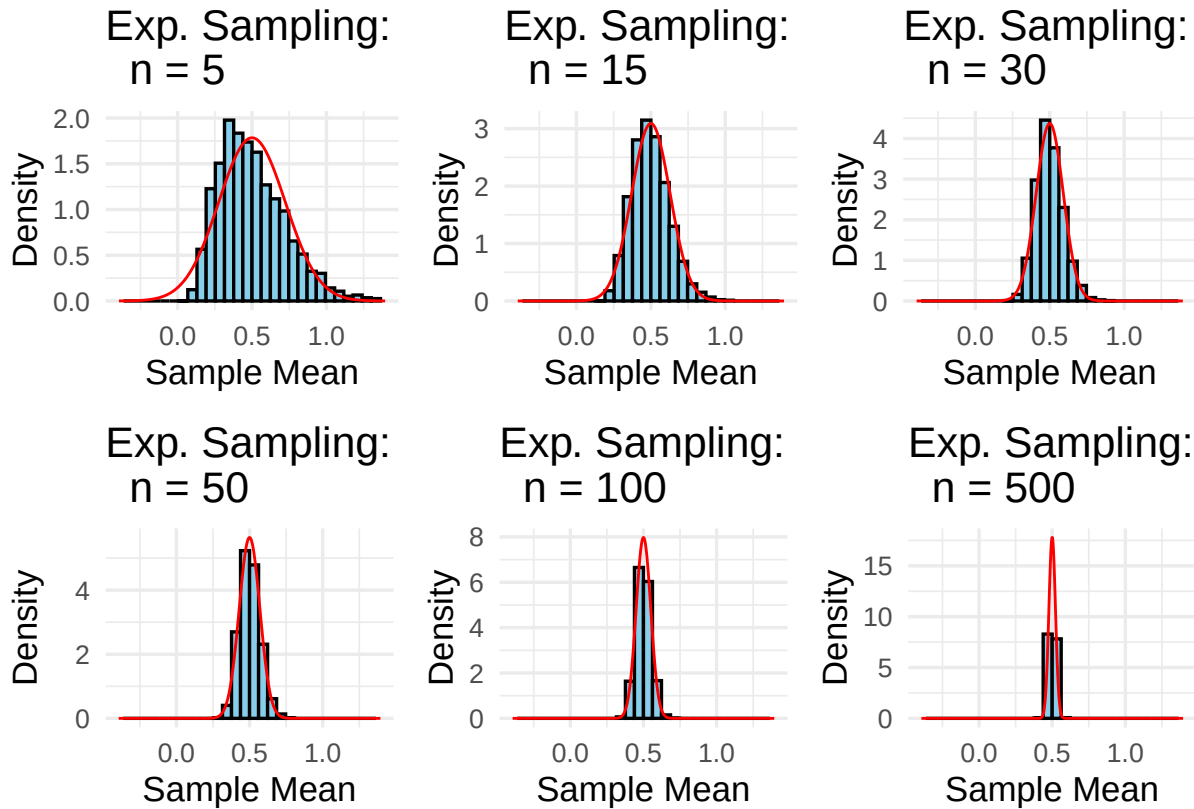
  # theoretical normal overlay
  theoretical <- data.frame(x = seq(xmin, xmax, length.out = 500))
  theoretical$y <- dnorm(theoretical$x, mean = mu, sd = sigma / sqrt(n))

  p <- ggplot(df, aes(x = mean)) +
    geom_histogram(aes(y =after_stat(density)),
      bins = 30,
      fill = "skyblue",
      color = "black") +
    geom_line(data = theoretical, aes(x = x, y = y),
      color = "red", linewidth = 0.5) +
    xlim(xmin, xmax) +
    labs(title = paste("Exp. Sampling:\n n =", n),
      x = "Sample Mean",
      y = "Density") +
    theme_minimal(base_size = 13)

  plots[[i]] <- p
}

# 2 x 3 grid
(plots[[1]] | plots[[2]] | plots[[3]]) /
(plots[[4]] | plots[[5]] | plots[[6]])

```



This demo illustrates the Central Limit Theorem (CLT). The CLT says that the standardized version of the sampling distribution converges to $N(0, 1)$

$$\sqrt{n} \left(\frac{X_n - \mu}{\sigma} \right) \xrightarrow{d} N(0, 1)$$

or equivalently,

$$\frac{X_n - \mu}{\sigma/\sqrt{n}} \xrightarrow{d} N(0, 1) \text{ as } n \rightarrow \infty$$

Imprecise $\rightarrow$ precise

- Confidence intervals get narrower with larger samples.
- Likelihood surfaces get steeper around the true value with larger samples
- Score fluctuations get smaller with larger samples

For MLEs

- With more data, log-likelihood curves concentrate around the truth.
- Score function cross zero close to the true parameter
- MLE variance approaches the Cramér-Rao bound (efficiency) in large samples.

Bias vs. Consistency

Just because something is unbiased, it isn't necessarily consistent. Consider for example an iid sample, $X_1, X_2, \dots, X_n$ from a $N(0, 1)$ distribution.

$\mathbb{E}(X_1) = 0 \rightarrow$ unbiased.

However, $\mathbb{P}(|X_1 - \mu| > \varepsilon)$ does not go to zero as $n \rightarrow \infty$. Why? Because X_1 doesn't change with sample size – it's fixed.

The biased version of the sample variance (i.e. denominator n),

$$s^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$$

is biased, but consistent.

$$\mathbb{E}(s^2) = \frac{n-1}{n} \sigma^2 = \sigma^2 - \frac{1}{n} \sigma^2 \neq \sigma^2$$

$$\text{Bias}(s^2) = -\frac{1}{n} \sigma^2$$

It's biased downward.

But, as $n \rightarrow \infty$, $\frac{n-1}{n} \rightarrow 1$.

So, $\mathbb{E}(s^2) \rightarrow \sigma^2$

In summary, “Unbiased” is a small-sample property; “consistent” is a large-sample property.

Bias vs. Consistency of the Variance Estimators

To highlight the difference between unbiasedness and consistency, the following image compares two common estimators for the variance of a sample.

$$S^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2 ; s^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$$

- Both estimators target the same parameter (i.e. population variance, σ^2),
- They behave quite differently depending on the sample size.

Demo: Samples were drawn from a heavy-tailed distribution (t distribution with 3 degrees of freedom) using a range of sample sizes. Monte-Carlo samples were obtained and each estimate of the variance was computed. The true variance (dashed line) is 3. The following plot shows the mean of each of the above estimators for different sample sizes.

The figure highlights the key distinction between unbiasedness and consistency. Unbiasedness is a finite-sample property: an estimator is centered at the true parameter for every fixed sample size. Consistency, in contrast, is an asymptotic property: as the sample size increases, the estimator converges to the true value. Thus, an unbiased estimator is correct on average even in small samples, whereas a biased estimator may deviate from the target when n is small but can still approach the true variance as n becomes large.

```
library(tidyverse)
```

```
set.seed(123)
```

```
# True variance of t(df=3)
```

```

true_var <- 3 / (3 - 2)

# Sample sizes
n_values <- c(5, 10, 20, 50, 100, 200, 500, 1000, 2000)

# Number of Monte Carlo simulations
B <- 20000

results <- map_df(n_values, function(n) {
  # Generate B simulated samples (each column = one sample)
  sims <- matrix(rt(n * B, df = 3), nrow = n, ncol = B)

  # Estimators
  s2_unbiased <- apply(sims, 2, var) # 1/(n-1)
  s2_mle <- colMeans((sims - colMeans(sims))^2) # 1/n

  tibble(
    n = n,
    mean_unbiased = mean(s2_unbiased),
    mean_mle = mean(s2_mle)
  )
})

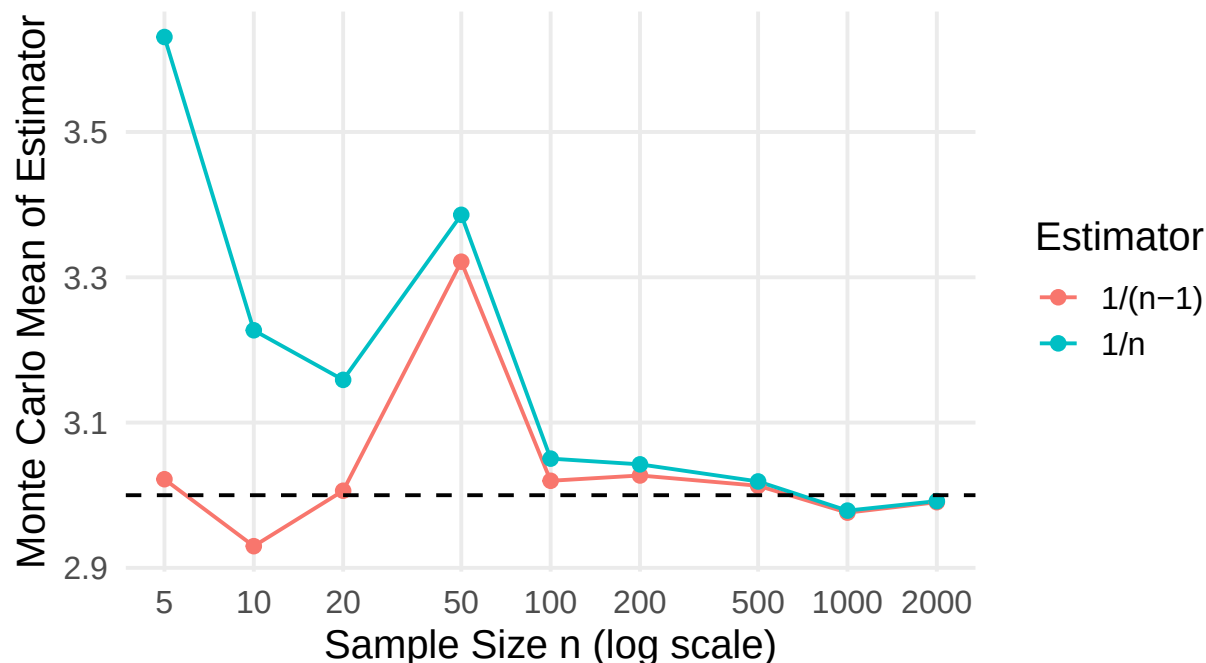
# Convert to long format for ggplot
df_long <- results %>%
  pivot_longer(
    cols = starts_with("mean_"),
    names_to = "Estimator",
    values_to = "Mean"
  ) %>%
  mutate(
    Estimator = recode(Estimator,
      mean_unbiased = "1/(n-1)",
      mean_mle = "1/n")
  )

# Plot
ggplot(df_long, aes(x = n, y = Mean, color = Estimator)) +
  geom_line(linewidth = 1.1) +
  geom_point(size = 2) +
  scale_x_log10(breaks = n_values) +
  geom_hline(yintercept = true_var, linetype = "dashed") +
  labs(
    title = "Convergence of Variance Estimators (t(df=3) Data) \n
      (true variance is 3)",
    x = "Sample Size n (log scale)",
    y = "Monte Carlo Mean of Estimator"
  ) +
  theme_minimal(base_size = 15) +
  theme(
    legend.position = "right",
    panel.grid.minor = element_blank()
  )

```


Convergence of Variance Estimators (t(df=3) Data)

(true variance is 3)



This plot may initially seem surprising. We are accustomed to thinking of the $1/(n-1)$ estimator as unbiased and the $1/n$ estimator as biased downward, so we might expect the $1/n$ curve to always lie below the $1/(n-1)$ curve. That expectation is correct in terms of theoretical means. However, here the data come from a t -distribution with 3 degrees of freedom, which is heavy-tailed and lacks a finite fourth moment. As a result, the finite-sample behavior can look erratic in simulation.

For this example,

- The $1/(n-1)$ estimator remains unbiased because the variance of $t(3)$ exists.
- The MLE ($1/n$) estimator is biased downward for every finite n .
- Because the fourth moment does not exist, the sampling distribution of the sample variance is extremely unstable, so Monte Carlo averages can fluctuate substantially.
- The heavy tails make the estimator highly variable in small samples, even though its expectation is well defined.

Main Point: The unbiased estimator is centered at the true variance for every fixed n , while the MLE is biased downward; however, both estimators are consistent and converge to the true variance as the sample size increases.

Checklist

Here is a checklist to decide if an estimator is likely consistent.

1. Is it based on averages? If yes, then the LLN applies and it's usually consistent.
2. Does the estimator solve an equation that involves averages? An example of this would be the MLE score equation. If yes, then likely consistent.
3. Is the parameter identifiable? If yes, then likely consistent.
4. Does the sample size show up in a way that shrinks the variance? If yes, then likely consistent.

Summary

Consistency

- Represents the long-run accuracy
- Converges in probability to the true parameter value
- Driven by LLN

Asymptotic Normality

- Sampling distribution is approximately normal for large n
- Allows Wald intervals and likelihood based inference
- Driven by CLT

Bottom Line: More data → curves sharpen, noise averages out and estimators stabilize.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.ticker import AutoMinorLocator

# -----
# Reproducibility + parameters
# -----
np.random.seed(123)

mu      = 50
sigma   = 1
B       = 5000          # number of simulations
n_values = [5, 15, 30, 50, 100, 500]

# Common x-axis limits (based on worst case: n=5)
sd_n5 = sigma / np.sqrt(5)
xmin = mu - 4 * sd_n5
xmax = mu + 4 * sd_n5

# -----
# Create 2x3 grid of plots
# -----
```

```

fig, axes = plt.subplots(2, 3, figsize=(14, 8), sharex=True, sharey=False)
axes = axes.flatten() # makes indexing easier

for idx, n in enumerate(n_values):
    ax = axes[idx]

    # Simulate B sample means
    sample_means = np.mean(
        np.random.normal(loc=mu, scale=sigma, size=(B, n)),
        axis=1
    )

    # Histogram + KDE
    sns.histplot(sample_means, bins=40, stat="density",
        color="skyblue", edgecolor="black", alpha=0.7, ax=ax)

    # Overlay smooth density (KDE)
    sns.kdeplot(sample_means, color="red", linewidth=2, ax=ax)

    # Titles and labels
    ax.set_title(f"n = {n}", fontsize=15, pad=10)
    ax.set_xlabel("Sample Mean" if idx >= 3 else "") # only bottom row
    ax.set_ylabel("Density")

    # Fixed x-limits for fair visual comparison
    ax.set_xlim(xmin, xmax)

    # Nice touches
    ax.grid(True, alpha=0.3)
    ax.xaxis.set_minor_locator(AutoMinorLocator())
    ax.tick_params(axis='both', which='major', labelsize=11)

## <Axes: ylabel='Density'>
## <Axes: ylabel='Density'>
## Text(0.5, 1.0, 'n = 5')
## Text(0.5, 0, '')
## Text(0, 0.5, 'Density')
## (48.21114561800017, 51.78885438199983)
## <Axes: ylabel='Density'>
## <Axes: ylabel='Density'>
## Text(0.5, 1.0, 'n = 15')
## Text(0.5, 0, '')
## Text(0, 0.5, 'Density')
## (48.21114561800017, 51.78885438199983)
## <Axes: ylabel='Density'>
## <Axes: ylabel='Density'>
## Text(0.5, 1.0, 'n = 30')
## Text(0.5, 0, '')
## Text(0, 0.5, 'Density')
## (48.21114561800017, 51.78885438199983)
## <Axes: ylabel='Density'>
## <Axes: ylabel='Density'>
## Text(0.5, 1.0, 'n = 50')
## Text(0.5, 0, 'Sample Mean')

```

```

## Text(0, 0.5, 'Density')
## (48.21114561800017, 51.78885438199983)
## <Axes: ylabel='Density'>
## <Axes: ylabel='Density'>
## Text(0.5, 1.0, 'n = 100')
## Text(0.5, 0, 'Sample Mean')
## Text(0, 0.5, 'Density')
## (48.21114561800017, 51.78885438199983)
## <Axes: ylabel='Density'>
## <Axes: ylabel='Density'>
## Text(0.5, 1.0, 'n = 500')
## Text(0.5, 0, 'Sample Mean')
## Text(0, 0.5, 'Density')
## (48.21114561800017, 51.78885438199983)

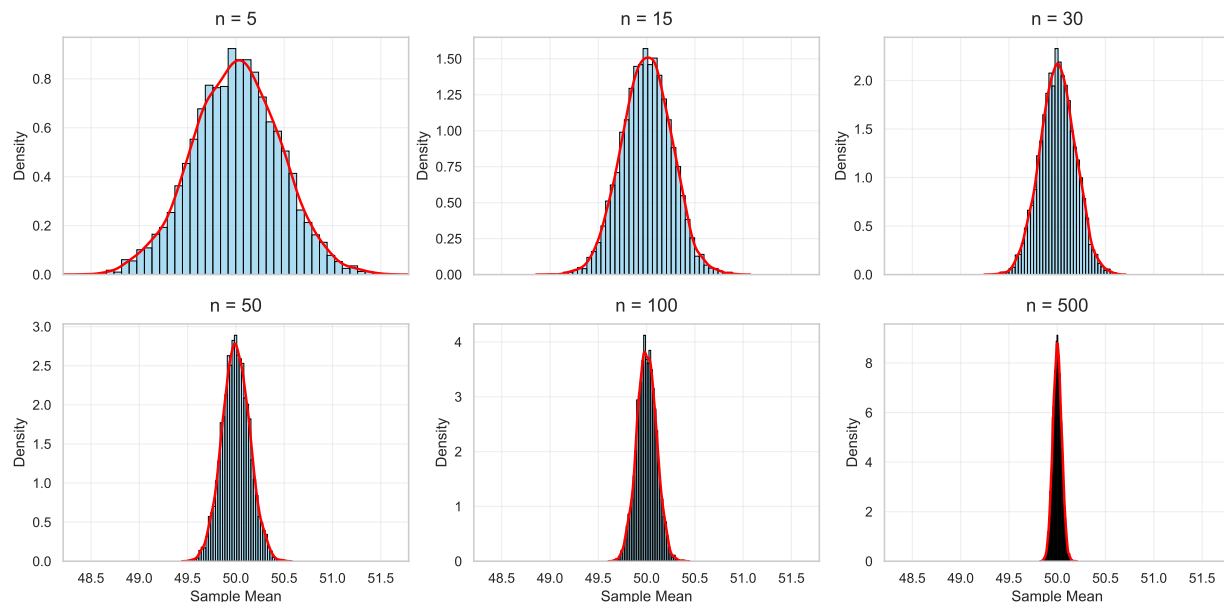
```

```

# Tight layout with proper spacing
plt.suptitle("Sampling Distribution of the Sample Mean\n"
            "as Sample Size Increases (B = 5,000 simulations each)",
            fontsize=18, y=1.02)
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

```

Sampling Distribution of the Sample Mean
as Sample Size Increases (B = 5,000 simulations each)



```

import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
import seaborn as sns
from matplotlib.ticker import AutoMinorLocator

# -----
# Reproducibility + parameters
# -----
np.random.seed(123)

```

```

lambda_exp = 2.0 # rate
mu = 1 / lambda_exp # mean = 0.5
sigma = 1 / lambda_exp # sd = 0.5
B = 5000 # number of simulations
n_values = [5, 15, 30, 50, 100, 500]

# Common x-axis limits (based on worst case: n=5)
sd_n5 = sigma / np.sqrt(5)
xmin = mu - 4 * sd_n5
xmax = mu + 4 * sd_n5

# -----
# Create 2x3 grid of plots
# -----
fig, axes = plt.subplots(2, 3, figsize=(14, 8), sharex=True, sharey=False)
axes = axes.flatten() # makes indexing easier

for idx, n in enumerate(n_values):
    ax = axes[idx]

    # Simulate B sample means from Exponential(rate=lambda)
    # Note: np.random.exponential(scale=1/lambda) = scale=mu
    sample_means = np.mean(
        np.random.exponential(scale=mu, size=(B, n)),
        axis=1
    )

    # Histogram
    sns.histplot(sample_means, bins=30, stat="density",
                 color="skyblue", edgecolor="black", alpha=0.7, ax=ax)

    # Theoretical normal overlay:  $N(\mu, \sigma^2 / n)$ 
    x_theory = np.linspace(xmin, xmax, 500)
    y_theory = norm.pdf(x_theory, loc=mu, scale=sigma / np.sqrt(n))
    ax.plot(x_theory, y_theory, color="red", linewidth=2)

    # Titles and labels
    ax.set_title(f"Exp. Sampling:\n n = {n}", fontsize=15, pad=10)
    ax.set_xlabel("Sample Mean" if idx >= 3 else "") # only bottom row
    ax.set_ylabel("Density")

    # Fixed x-limits for fair visual comparison
    ax.set_xlim(xmin, xmax)

    # Nice touches
    ax.grid(True, alpha=0.3)
    ax.xaxis.set_minor_locator(AutoMinorLocator())
    ax.tick_params(axis='both', which='major', labelsize=11)

## <Axes: ylabel='Density'>
## [<matplotlib.lines.Line2D object at 0x000002D975823E30>]
## Text(0.5, 1.0, 'Exp. Sampling:\n n = 5')
## Text(0.5, 0, '')

```

```

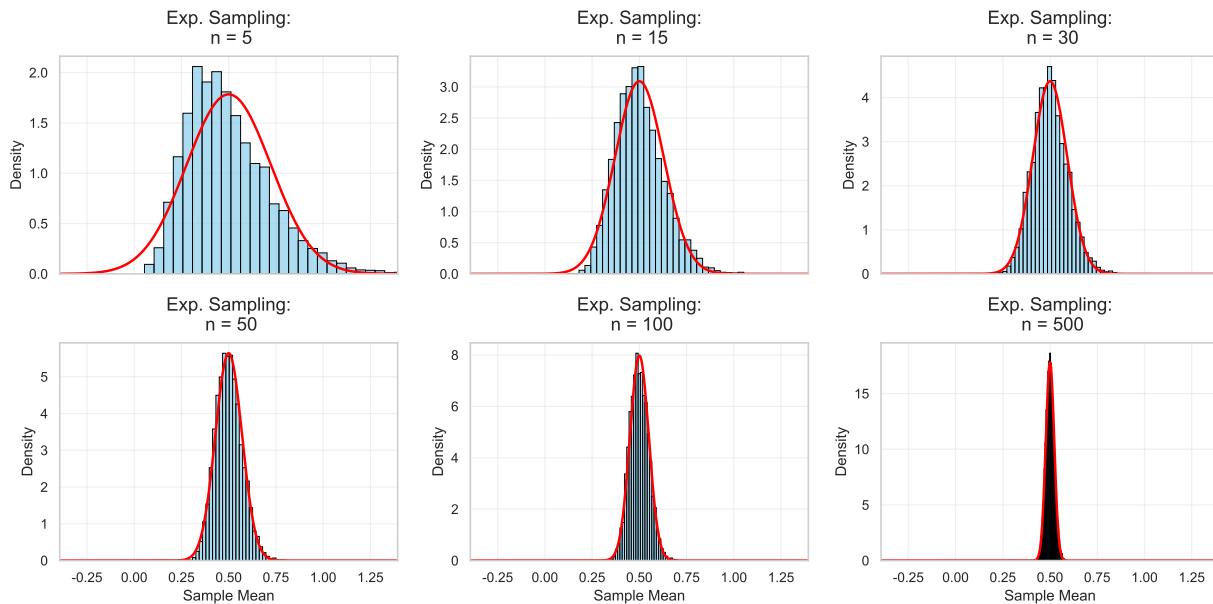
## Text(0, 0.5, 'Density')
## (-0.39442719099991586, 1.3944271909999157)
## <Axes: ylabel='Density'>
## [<matplotlib.lines.Line2D object at 0x000002D9752552B0>]
## Text(0.5, 1.0, 'Exp. Sampling:\n n = 15')
## Text(0.5, 0, '')
## Text(0, 0.5, 'Density')
## (-0.39442719099991586, 1.3944271909999157)
## <Axes: ylabel='Density'>
## [<matplotlib.lines.Line2D object at 0x000002D975B7F500>]
## Text(0.5, 1.0, 'Exp. Sampling:\n n = 30')
## Text(0.5, 0, '')
## Text(0, 0.5, 'Density')
## (-0.39442719099991586, 1.3944271909999157)
## <Axes: ylabel='Density'>
## [<matplotlib.lines.Line2D object at 0x000002D975BEC140>]
## Text(0.5, 1.0, 'Exp. Sampling:\n n = 50')
## Text(0.5, 0, 'Sample Mean')
## Text(0, 0.5, 'Density')
## (-0.39442719099991586, 1.3944271909999157)
## <Axes: ylabel='Density'>
## [<matplotlib.lines.Line2D object at 0x000002D975C27740>]
## Text(0.5, 1.0, 'Exp. Sampling:\n n = 100')
## Text(0.5, 0, 'Sample Mean')
## Text(0, 0.5, 'Density')
## (-0.39442719099991586, 1.3944271909999157)
## <Axes: ylabel='Density'>
## [<matplotlib.lines.Line2D object at 0x000002D975D27DA0>]
## Text(0.5, 1.0, 'Exp. Sampling:\n n = 500')
## Text(0.5, 0, 'Sample Mean')
## Text(0, 0.5, 'Density')
## (-0.39442719099991586, 1.3944271909999157)

# Overall title
plt.suptitle("Sampling Distribution of the Sample Mean from Exponential\n"
             "as Sample Size Increases (B = 5,000 simulations each)",
             fontsize=18, y=1.02)

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

```

Sampling Distribution of the Sample Mean from Exponential
as Sample Size Increases (B = 5,000 simulations each)



```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

np.random.seed(123)

# True variance of  $t(df=3)$ 
true_var = 3 / (3 - 2)

# Sample sizes
n_values = [5, 10, 20, 50, 100, 200, 500, 1000, 2000]

# Number of Monte Carlo simulations
B = 20000

records = []

for n in n_values:
    # Generate  $t(df=3)$  samples for all B simulations
    sims = np.random.standard_t(df=3, size=(n, B))

    # unbiased estimator ( $1/(n-1)$ )
    s2_unbiased = sims.var(axis=0, ddof=1)

    # MLE estimator ( $1/n$ ): mean squared deviation from sample mean
    means = sims.mean(axis=0)
    s2_mle = ((sims - means)**2).mean(axis=0)

    records.append({
        "n": n,
```

```

        "mean_unbiased": s2_unbiased.mean(),
        "mean_mle": s2_mle.mean()
    })

# Convert to DataFrame
results = pd.DataFrame(records)

# Long format for plotting
df_long = results.melt(
    id_vars="n",
    value_vars=["mean_unbiased", "mean_mle"],
    var_name="Estimator",
    value_name="Mean"
)

df_long["Estimator"] = df_long["Estimator"].replace({
    "mean_unbiased": "1/(n-1)",
    "mean_mle": "1/n"
})

# Plot
plt.figure(figsize=(10, 6))
sns.lineplot(data=df_long, x="n", y="Mean", hue="Estimator", marker="o")

plt.xscale("log")
plt.axhline(true_var, linestyle="--")

plt.title("Convergence of Variance Estimators (t(df=3) Data)\n(true variance is 3)")
plt.xlabel("Sample Size n (log scale)")
plt.ylabel("Monte Carlo Mean of Estimator")

plt.grid(True, which="major", axis="both")
plt.show()

```